

```

        return true; // Found the exit, return true.
    }

    maze[y][x] = PATH; // Mark the path in the maze.
    ❷ visited.push(String(x) + "," + String(y));
    ❸ //printMaze(maze) // Uncomment to view each forward step.

    // Explore the north neighboring point:
    if ((y + 1 < HEIGHT) && ((maze[y + 1][x] == EMPTY) ||
        (maze[y + 1][x] == EXIT)) &&
        (visited.indexOf(String(x) + "," + String(y + 1)) == -1)) {
        // RECURSIVE CASE
        if (solveMaze(maze, x, y + 1, visited)) {
            return true; // BASE CASE
        }
    }

    // Explore the south neighboring point:
    if ((y - 1 >= 0) && ((maze[y - 1][x] == EMPTY) ||
        (maze[y - 1][x] == EXIT)) &&
        (visited.indexOf(String(x) + "," + String(y - 1)) == -1)) {
        // RECURSIVE CASE
        if (solveMaze(maze, x, y - 1, visited)) {
            return true; // BASE CASE
        }
    }

    // Explore the east neighboring point:
    if ((x + 1 < WIDTH) && ((maze[y][x + 1] == EMPTY) ||
        (maze[y][x + 1] == EXIT)) &&
        (visited.indexOf(String(x + 1) + "," + String(y)) == -1)) {
        // RECURSIVE CASE
        if (solveMaze(maze, x + 1, y, visited)) {
            return true; // BASE CASE
        }
    }

    // Explore the west neighboring point:
    if ((x - 1 >= 0) && ((maze[y][x - 1] == EMPTY) ||
        (maze[y][x - 1] == EXIT)) &&
        (visited.indexOf(String(x - 1) + "," + String(y)) == -1)) {
        // RECURSIVE CASE
        if (solveMaze(maze, x - 1, y, visited)) {
            return true; // BASE CASE
        }
    }

    maze[y][x] = EMPTY; // Reset the empty space.

```